

```

1  import sounddevice as sd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy.signal import butter, filtfilt, find_peaks
5  from scipy.ndimage import gaussian_filter1d
6  import csv
7  import os
8  from datetime import datetime
9  from scipy.io.wavfile import write
10
11 # =====
12 # INPUT UTILISATEUR
13 # =====
14 nom = input("Nom de l'essai : ")
15 num_exp = input("Numéro de l'essai : ")
16
17 plt.close("all")
18
19 # =====
20 # PARAMÈTRES
21 # =====
22 FS = 48000
23 DURATION = 2
24 CHANNELS = 2
25
26 ACC_CHANNEL = 0
27 MIC_CHANNEL = 1
28
29 SENS_ACCEL = 0.01      # V/g
30 SENS_MIC = 0.05        # V/Pa
31 AUDIO_FULL_SCALE_V = 1.0
32
33 # =====
34 # ACQUISITION
35 # =====
36 print("Acquisition en cours...")
37
38 sd.default.device = 26 # Remplacez par l'index de votre interface audio (voir sd.query_devices())
39 data = sd.rec(
40     int(DURATION * FS),
41     samplerate=FS,
42     channels=CHANNELS,
43     dtype='float32'
44 )
45 sd.wait()
46 print("Acquisition terminée")
47
48 # =====
49 # SAUVEGARDE AUDIO
50 # =====
51
52 # Normalisation pour éviter clipping
53 audio_norm = data / np.max(np.abs(data))
54
55 # Conversion en int16 (format WAV standard)
56 audio_int16 = (audio_norm * 32767).astype(np.int16)
57
58
59
60 # =====
61 # EXTRACTION
62 # =====
63 acc_signal = data[:, ACC_CHANNEL]
64 mic_signal = data[:, MIC_CHANNEL]
65
66 # =====
67 # FILTRE PASSE-HAUT
68 # =====
69 def highpass(sig, fs, cutoff=1):
70     b, a = butter(2, cutoff/(fs/2), btype='high')
71     return filtfilt(b, a, sig)
72
73 acc_signal = highpass(acc_signal, FS)
74 mic_signal = highpass(mic_signal, FS)
75
76 # =====
77 # CONVERSIONS
78 # =====
79 acc_voltage = acc_signal * AUDIO_FULL_SCALE_V
80 mic_voltage = mic_signal * AUDIO_FULL_SCALE_V
81
82 acc_g = acc_voltage / SENS_ACCEL
83 pressure = mic_voltage / SENS_MIC
84
85 # =====
86 # NORMALISATION
87 # =====

```

```

88     def normalize(sig):
89         max_val = np.max(np.abs(sig))
90         return sig if max_val == 0 else sig / max_val
91
92     acc_g = normalize(acc_g)
93     pressure = normalize(pressure)
94
95     # =====
96     # TEMPS
97     # =====
98     N = len(acc_g)
99     t = np.arange(N) / FS
100
101     # =====
102     # FFT
103     # =====
104     window = np.hanning(N)
105
106     acc_fft = np.abs(np.fft.rfft(acc_g * window)) / N
107     mic_fft = np.abs(np.fft.rfft(pressure * window)) / N
108
109     freq = np.fft.rfftfreq(N, 1/FS)
110
111     # =====
112     # LISSAGE
113     # =====
114     sigma = 10
115     acc_fft = gaussian_filter1d(acc_fft, sigma)
116     mic_fft = gaussian_filter1d(mic_fft, sigma)
117
118     # =====
119     # PASSAGE dB
120     # =====
121     acc_fft_db = 20 * np.log10(acc_fft + 1e-12)
122     mic_fft_db = 20 * np.log10(mic_fft + 1e-12)
123
124     # =====
125     # DÉTECTION DES PICS
126     # =====
127     min_height_db = -80
128     min_distance = 20
129     prominence = 5
130
131     # Accéléromètre
132     peaks_acc, _ = find_peaks(
133         acc_fft_db,
134         height=min_height_db,
135         distance=min_distance,
136         prominence=prominence
137     )
138
139     sorted_acc = sorted(peaks_acc, key=lambda p: acc_fft_db[p], reverse=True)
140
141     print("\nPics Accéléromètre :")
142     for p in sorted_acc[:10]:
143         if freq[p] < 3000:
144             print(f"{freq[p]:.1f} Hz | {acc_fft_db[p]:.1f} dB")
145
146     # Micro
147     peaks_mic, _ = find_peaks(
148         mic_fft_db,
149         height=min_height_db,
150         distance=min_distance,
151         prominence=prominence
152     )
153
154     sorted_mic = sorted(peaks_mic, key=lambda p: mic_fft_db[p], reverse=True)
155
156     print("\nPics Microphone :")
157     for p in sorted_mic[:10]:
158         if freq[p] < 3000:
159             print(f"{freq[p]:.1f} Hz | {mic_fft_db[p]:.1f} dB")
160
161     # =====
162     # ZOOM TEMPOREL
163     # =====
164     delta_t = 0.05
165     peak_index = np.argmax(np.abs(acc_g))
166     delta_n = int(delta_t * FS)
167
168     start = max(0, peak_index - delta_n)
169     end = min(N, peak_index + delta_n)
170
171     t_zoom = t[start:end]
172     acc_zoom = acc_g[start:end]
173     mic_zoom = pressure[start:end]
174
175     # =====

```

```

176 # GRAPHES
177 # =====
178 fig, ax = plt.subplots(2, 1, figsize=(12,7))
179
180 # TEMPS
181 ax[0].plot(t_zoom, acc_zoom, label="Accéléromètre", color="blue")
182 ax[0].plot(t_zoom, mic_zoom, label="Micro", color="red")
183 ax[0].set_title("Signal temporel")
184 ax[0].grid()
185 ax[0].legend()
186
187 # FFT LOG
188 ax[1].plot(freq, acc_fft_db, label="Accéléromètre", color="blue")
189 ax[1].plot(freq, mic_fft_db, label="Micro", color="red")
190
191 # AJOUT DES PICS
192 ax[1].plot(freq[sorted_acc[:10]], acc_fft_db[sorted_acc[:10]], "bx")
193 ax[1].plot(freq[sorted_mic[:10]], mic_fft_db[sorted_mic[:10]], "rx")
194
195 ax[1].set_xlim(10, 2000)
196 ax[1].set_title("FFT (log)")
197 ax[1].grid(which="both")
198 ax[1].legend()
199
200
201 plt.tight_layout()
202
203 plt.show()
204
205 if input("Garder ?") == 'Y':
206     # Sauvegarde
207     filename = f"{nom}_{datetime.now().strftime('%Y-%m-%d_%H-%M-%S')}.png"
208     fig.savefig(filename, dpi=300)
209     audio_filename = f"{nom}_{datetime.now().strftime('%Y-%m-%d_%H-%M-%S')}.wav"
210     write(audio_filename, FS, audio_int16)
211
212     print(f"Fichier audio sauvegardé : {audio_filename}")
213
214     # =====
215     # SAUVEGARDE CSV
216     # =====
217     filename = "pics.csv"
218     file_exists = os.path.isfile(filename)
219
220     with open(filename, mode='a', newline='') as file:
221         writer = csv.writer(file)
222
223         if not file_exists:
224             writer.writerow([
225                 "nom",
226                 "timestamp",
227                 "num_exp",
228                 "frequence_hz",
229                 "amplitude_db"
230             ])
231
232         now = datetime.now().isoformat()
233
234         # ACC
235         for p in sorted_acc[:10]:
236             if freq[p] < 3000:
237                 writer.writerow([now, nom, num_exp, "acc", freq[p], acc_fft_db[p]])
238
239         # MIC
240         for p in sorted_mic[:10]:
241             if freq[p] < 3000:
242                 writer.writerow([now, nom, num_exp, "mic", freq[p], mic_fft_db[p]])
243
244     print("Pics FFT enregistrés dans pics.csv")

```